```
class interrupt_flag
{
public:
    void set();
    bool is_set() const;
};
thread_local interrupt_flag this_thread_interrupt_flag;     <-- 1

class interruptible_thread
{
    std::thread internal_thread;

    interrupt_flag* flag;
public:
    template<typename FunctionType>
    interruptible_thread(FunctionType f)
    {
        std::promise<interrupt_flag*> p;      <-- 2
        internal_thread=std::thread([f,&p]{                 <-- 3
                p.set_value(&this_thread_interrupt_flag);
                f();                                        <-- 4
            });
        flag=p.get_future().get();     <-- 5
    }
    void interrupt()
    {
        if(flag)
        {
            flag->set();     <-- 6
        }
    }
};
```

The supplied function `f` is wrapped in a lambda function ❸ that holds a copy of `f` and a reference to the local promise `p` ❷. The lambda sets the value of the promise to the address of the `this_thread_interrupt_flag` (which is declared `thread_local` ❶) for the new thread before invoking the copy of the supplied function ❹. The calling thread then waits for the future associated with the promise to become ready and stores the result in the `flag` member variable ❺. Note that even though the lambda is running on the new thread and has a dangling reference to the local variable `p`, this is OK because the